REPORT

On

COMPUTER NETWORKS LAB (Course Code: V23CSL09)

B. TECH V Semester (2024-2028 Batch)

Academic Year: 2026-27



SRI VASAVI ENGINEERING COLLEGE(Autonomous)

Pedatadepalli, Tadepalligudem-534101

Department of Computer Science & Engineering

SRI VASAVI ENGINEERING COLLEGE(Autonomous)
PEDATADEPALLI, TADEPALLIGUDEM.

Certificate

This is to certify _____ bearing Roll No. _____ of Computer Science and Technology Branch of V Semester submitted the report on as a part of COMPUTER NETWORKS LAB (Course Code: V23CSL09) during the academic year 2026-2027.

No. of Experiment done:

Faculty In Charge

Head of the Department

Examiner 1

Examiner 2

INDEX

S.No	Date	Name of the Experiment	Page No.	Remarks
1.		Study of Network devices in detail and connect the computers in Local Area Network.		
2.		Develop a Program to implement the data link layer framing methods such as i) Character stuffing Bit stuffing.		
3.		Develop a Program to implement data link layer farming method checksum.		
4.		Develop a program for Hamming Code generation for error detection and correction.		
5.		Develop a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.		
6.		Develop a Program to implement Sliding window protocol for Goback-N.		
7.		Develop a Program to implement Sliding window protocol for Selective repeat.		
8.		Develop a Program to implement Stop and Wait Protocol.		
9.		Develop a program for congestion control using leaky bucket algorithm		

10.		Develop a Program to implement Dijkstra's algorithm to compute the Shortest path through a graph.		
11.		Develop a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).		
12.		Develop a Program to Implement a Broadcast Tree by Taking a Subnet of Hosts.		
13.		Wireshark i. Packet Capture Using Wire shark ii. Starting Wire shark iii. iii. Viewing Captured Traffic iv. Analysis and Statistics & Filters.		
14.		Discover active hosts, open ports and running services in a network using Nmap		
15.		Find Operating System in a host using Nmap		
16.		16. Do the Following Using NS2 Simulator: i. NS2 Simulator – Introduction ii. Simulate to Find the Number of Packets Dropped. iii. Simulate to Find the Number of Packets Dropped by TCP/UDP. iv. Simulate to Find the Number of Packets Dropped Due to Congestion.		

EXPERIMENT-1

AIM: Study of Network devices in detail and connect the computers in Local Area Network.

Title of the experiment:

Study of Network devices in detail and connect the computers in Local Area Network.

Objectives:

- To Study the Network devices in details
- Understanding the connecting the computers in Local Area Network.

Equipment/ Software Used:

- Computer system with internet access.
- Web browser (Google Chrome, Edge, Firefox).
- Search engines (Google, Bing).
- Reference websites (tutorials, documentation, Wikipedia, etc.).

Theory:

- **Computer Networks:**

  A computer network is a system in which multiple computers or devices are interconnected to share resources, exchange data, and communicate with each other. Networks are broadly classified into LAN (Local Area Network),
  MAN (Metropolitan Area Network), and WAN (Wide Area Network). Among these, LAN is widely used in computer labs, offices, and homes.

- **Network Devices:**

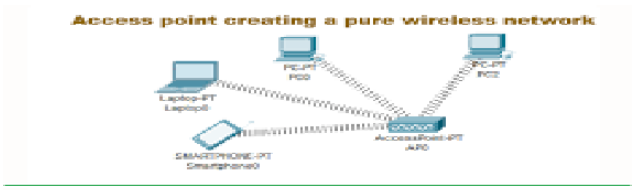
  To set up a network, certain hardware components are required, known as network devices. These devices make communication between systems possible by forwarding, routing, or broadcasting data. Common examples are Hub, Switch, Router, and NIC. Studying these devices helps us understand how data flows in a network and how LANs are established.

Procedure:

Different kinds of Network Devices:

1.Access Point:

Provides wireless connectivity to a wired network via wi-fi





Use At:

Homes, Offices, public Wi-fi Zones, Schools, Airports

1. Modem:

Converts digital signal to Analog and vice versa to provide internet access.



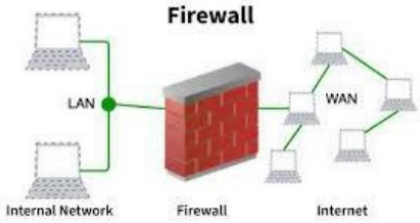
Use at:

Homes, ISPs (Internet Service Providers), Small Offices

2. Firewall:

Monitors and Controls network traffic to protect against threats.





Use at:

Entreprises, Data Centres, Cloud Environments, Homes (via Router Firewall).

3. Hub:

Broadcasts data to all connected devices without filtering.



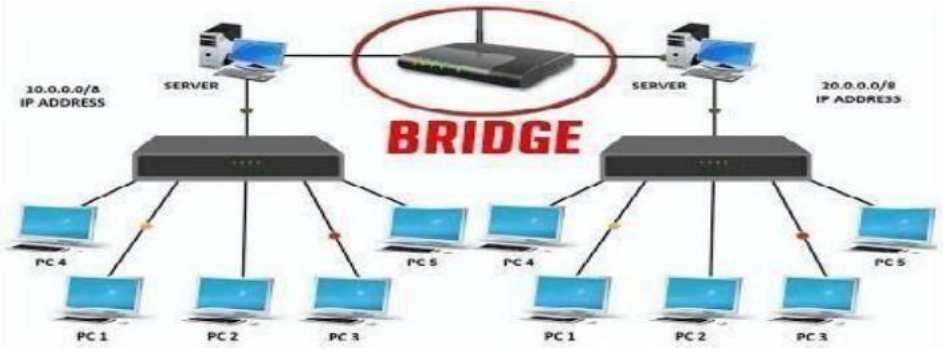


Use at:

Legacy or Small networks (rarely used today due to in Efficiency).

4. Bridge:

Connects and filters traffic between two network segments.



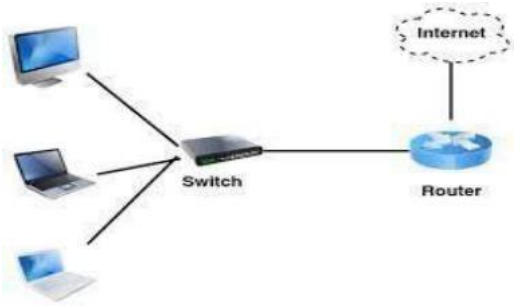
Use at:

Small to medium LANs, legacy systems, Network Segmentation.

5. Switch:

Forwards Data intelligently between on the same network.



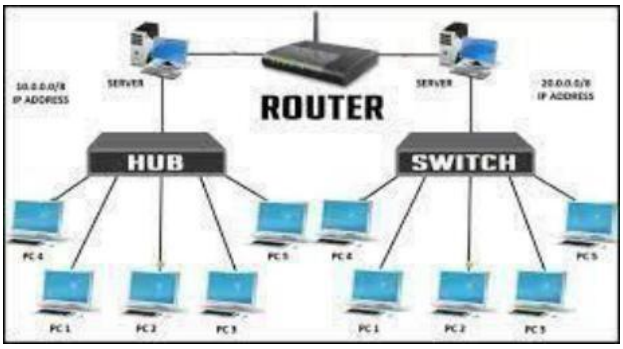


Use at:

Offices, Schools, Data Centres, Network Backbones

6. Router:

Directs data Between different Networks using IP addresses.

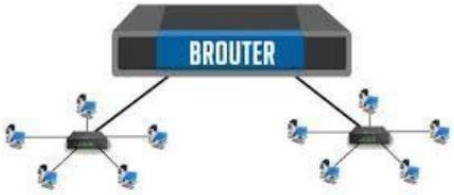


Use at:

Homes, Entreprises, ISPs, Backbone internet, Infrastructure.

7. Repeater :

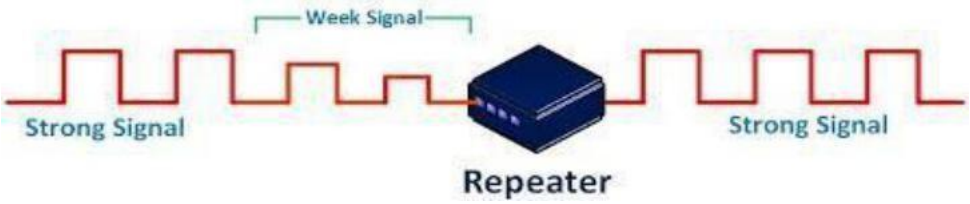
Amplifies weak signals to extend the range of the network.



Use at:

Long-distance LANs, Large buildings, Remote Office Locations

10. Gateway:

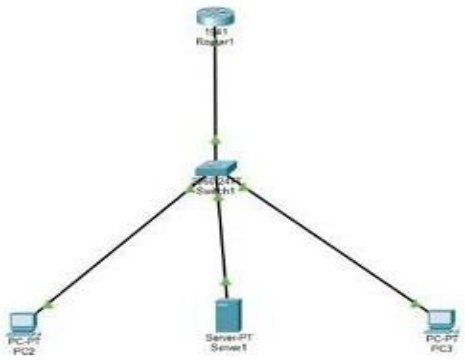


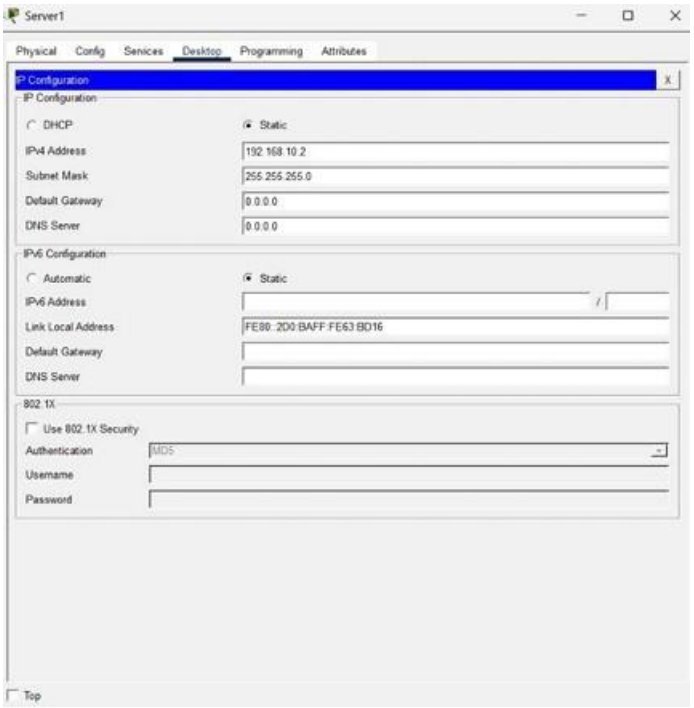
Connecting the Computers in LAN:

Steps :

Ensure all computers have NICs (Network Interface Card).

1. Use Ethernet cables to connect each computer to a switch.
2. Connect the switch to the Router if you want internet access.
3. Assign IP address to each computer- can be static or dynamic (Automatic using DHCP).
4. Check connectivity.




Server1
Physical
Config
Services
Desktop
Programming
Attributes
IP Configuration
IP Configuration
DHCP
Static
IPv4 Address
192.168.10.2
Subnet Mask
255.255.255.0
Default Gateway
0.0.0.0
DNS Server
0.0.0.0
IPv6 Configuration
Automatic
Static
IPv6 Address
Link Local Address
FE80::2D0:BAFF:FE63:BD16
Default Gateway
DNS Server
802.1X
Use 802.1X Security
Authentication
MD5
Username
Password
Top


Physical
Config
CLI
Attributes
IOS Command Line Interface
Copy
Paste

EXPERIMENT-2

AIM: Develop a Program to implement the data link layer framing methods such as i. Character stuffing. ii. Bit stuffing.

i. Character stuffing

Program:

```
#include <stdio.h>
#include <string.h>
int main() {
char s[100], a, b, out[200];
int i, j = 0;
printf("Enter characters to be stuffed: ");
scanf("%s", s);
printf("Enter a character that represents starting delimiter: ");
scanf(" %c", &a);
printf("Enter a character that represents ending delimiter: ");
scanf(" %c", &b);
out[j++] = a; out[j++] = a;
for(i = 0; s[i]; i++) {
out[j++] = s[i];
if(s[i] == a || s[i] == b) out[j++] = s[i];}
out[j++] = b; out[j] = '\0';
printf("After stuffing: %s\n", out);
return 0;
}
```

Output:

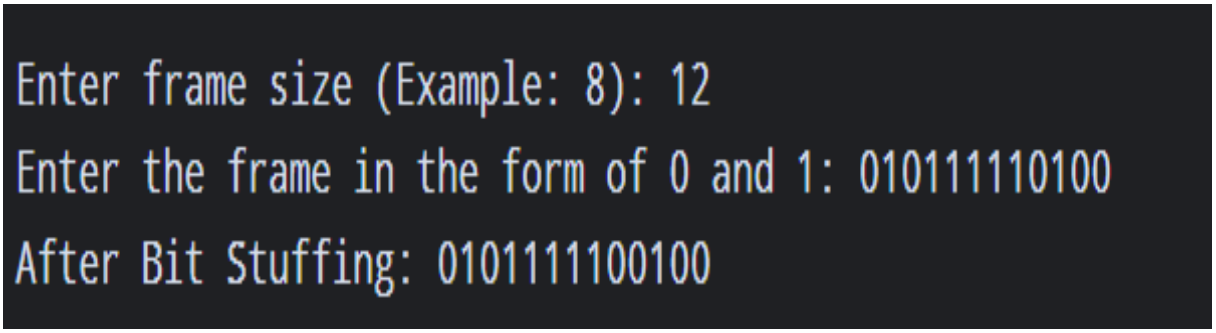
```
Enter characters to be stuffed: goodday
Enter a character that represents starting delimiter: d
Enter a character that represents ending delimiter: g
After stuffing: ddggooddddayg
```

ii. Bit stuffing

Program:

```
#include <stdio.h>
int main() {
int n, i, c = 0;
char s[100], b[200];
printf("Enter frame size (Example: 8): ");
scanf("%d", &n);
printf("Enter the frame in the form of 0 and 1: ");
scanf("%s", s);
for(i = 0; i < n; i++) {
b[c++] = s[i];
if(s[i] == '1' && i >= 4 && s[i-1]=='1' && s[i-2]=='1' && s[i-3]=='1' && s[i-4]=='1')
b[c++] = '0';
}
b[c] = '\0';
printf("After Bit Stuffing: %s\n", b);
return 0;
}
```

Output:



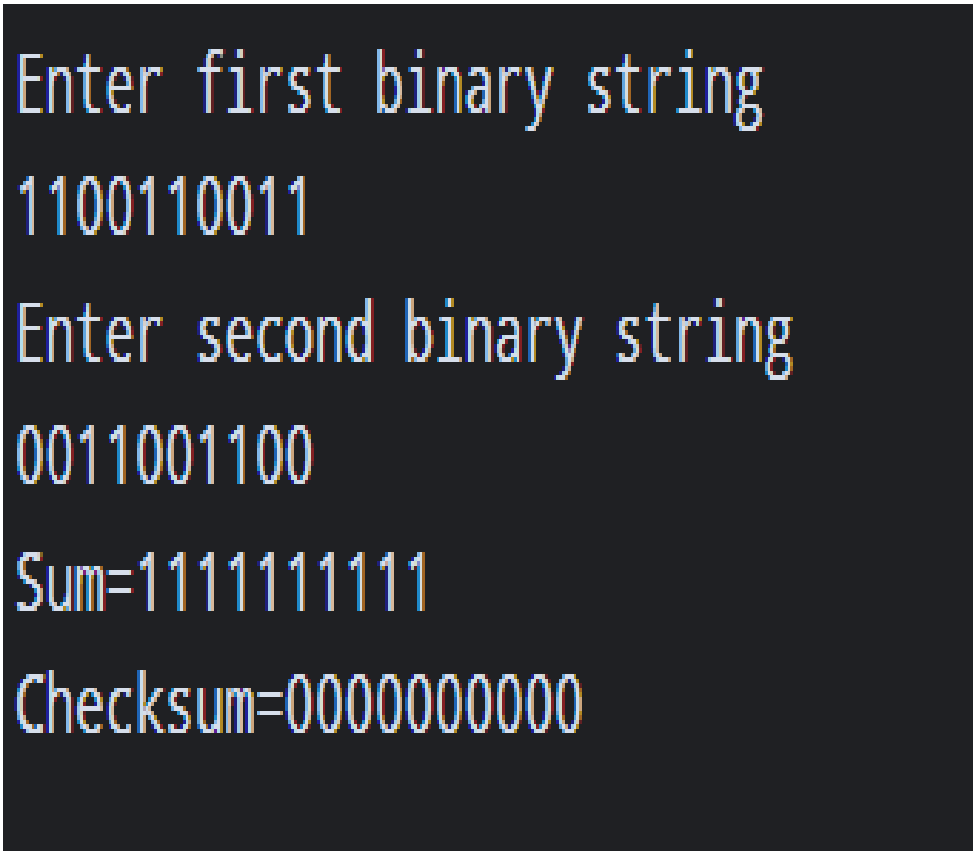
EXPERIMENT-3

AIM: Develop a Program to implement data link layer framing method checksum.

Program:

```
#include <stdio.h>
#include <string.h>
int main()
{
char a[50], b[50], sum[50];
int i, c = 0, n;
printf("Enter first binary string\n");
scanf("%s", a);
printf("Enter second binary string\n");
scanf("%s", b);
n = strlen(a);
for(i = n-1; i >= 0; i--)
{
int x = (a[i]-'0') + (b[i]-'0') + c;
sum[i] = x % 2 + '0';
c = x / 2;
}
sum[n] = '\0';
printf("Sum=%s\n", sum);
for(i = 0; i < n; i++)
sum[i] = sum[i] == '0' ? '1' : '0';
printf("Checksum=%s\n", sum);
return 0;
}
```

Output:


Enter first binary string
1100110011
Enter second binary string
0011001100
Sum=1111111111
Checksum=0000000000

EXPERIMENT-4

AIM: Develop a program for Hamming Code generation for error detection and correction

Program:

```
#include <stdio.h>
int main()
{
int d[4], r[3], c, i;
printf("Enter 4 bits of data one by one\n");
for(i=0;i<4;i++) scanf("%d",&d[i]);
r[0]=d[0]^d[1]^d[3];
r[1]=d[0]^d[2]^d[3];
r[2]=d[1]^d[2]^d[3];
printf("\nEncoded data is\n%d%d%d%d%d%d%d\n",
r[0],r[1],d[0],r[2],d[1],d[2],d[3]);
printf("\nEnter received data bits one by one\n");
int b[7];
for(i=0;i<7;i++) scanf("%d",&b[i]);
c=(b[0]^b[2]^b[4]^b[6])*1+
(b[1]^b[2]^b[5]^b[6])*2+
(b[3]^b[4]^b[5]^b[6])*4;
if(c)
{
printf("\nError at position %d\n",c);
b[c-1]^=1;
printf("Corrected data is\n");
for(i=0;i<7;i++) printf("%d",b[i]);
}
else printf("\nNo error while transmission of data\n");
return 0;
}
```

Output:

```
Enter 4 bits of data one by one
1
0
0
1

Encoded data is
0011001

Enter received data bits one by one
0
0
1
1
0
0
1

No error while transmission of data
```

EXPERIMENT-5

AIM:Develop a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.

Program:

```
#include <stdio.h>
#include <string.h>
char data[200], temp[200], gen[20], crc[20];
int n, m, i, j;
void divide() {
strcpy(temp, data);
for(i=0; i<n; i++)
if(temp[i]=='1')
for(j=0; j<m; j++)
temp[i+j] = (temp[i+j]==gen[j]) ? '0':'1';
for(i=0; i<m-1; i++) crc[i]=temp[n+i];
crc[m-1]='\0';
}
int main() {
int op;
while(1) {
printf("\n1.crc12\n2.crc16\n3.crc ccitt\n4.exit\nEnter your option:");
scanf("%d",&op);
if(op==4) break;
if(op==1) strcpy(gen,"1100000001111");
else if(op==2) strcpy(gen,"11000000000000101");
else if(op==3) strcpy(gen,"10001000000100001");
else { printf("Invalid option\n"); continue; }

printf("enter data:");
scanf("%s",data);
m=strlen(gen); n=strlen(data);
printf("-----------------------------\ngenerating polynomial:%s\n",gen);
for(i=0;i<m-1;i++) data[n+i]='0';
data[n+m-1]='\0'; n=strlen(data);
divide();
printf("-----------------------------\nmod-ified data is:%s\n",data);
printf("-----------------------------\nchecksum is:%s\n",crc);
```

```
strcat(data,crc);
printf("\nfinal codeword is : %s\n",data);

printf("----------------------------\ntest error detection 0(yes) 1(no):");
scanf("%d",&op);
if(op==1) printf("no error detected\n");
else printf("Error detected\n");
}
return 0;
}
```

Output:

```
1.crc12
2.crc16
3.crc ccitt
4.exit
Enter your option:2
enter data:1100110011001100
------------------------------
generating polynomial:11000000000000101
------------------------------
mod-ified data is:11001100110011000000000000000000
------------------------------
checksum is:101110111001011

final codeword is : 11001100110011000000000000000000101110111001011
------------------------------
test error detection 0(yes) 1(no):1
no error detected

1.crc12
2.crc16
3.crc ccitt
4.exit
Enter your option:4
```

EXPERIMENT-6

AIM: Develop a Program to implement Sliding window protocol for Goback N.

Program:

```
#include <stdio.h>
int main() {
int n, ack, i;
printf("enter window size\n");
scanf("%d", &n);
for(i=0; i<n; i++)
printf("Frame %d has been transmitted.\n", i);
while(1) {
printf("\nPlease enter the last Acknowledgement received.\n");
scanf("%d", &ack);
if(ack==n) break;
if(ack<0 || ack>n) {
printf("Invalid acknowledgement.\n");
continue;
}
for(i=ack; i<n; i++)
printf("Frame %d has been transmitted.\n", i);
}
return 0;
}
```

Output:

```
1.crc12
2.crc16
3.crc ccitt
4.exit
Enter your option:2
enter data:1100110011001100
----------------------------
generating polynomial:11000000000000101
----------------------------
mod-ified data is:11001100110011000000000000000000
----------------------------
checksum is:101110111001011

final codeword is : 11001100110011000000000000000000101110111001011
----------------------------
test error detection 0(yes) 1(no):1
no error detected

1.crc12
2.crc16
3.crc ccitt
4.exit
Enter your option:4
```

EXPERIMENT-7

AIM: Develop a Program to implement Sliding window protocol for Selective repeat.

Program:

```
#include <stdio.h>
int main() {
int w, n, i, j;
printf("Enter window size: ");
scanf("%d", &w);
printf("Enter number of frames to transmit: ");
scanf("%d", &n);
int f[n];
printf("Enter %d frames: ", n);
for(i=0; i<n; i++) scanf("%d", &f[i]);
printf("With sliding window protocol the frames will be sent in the following manner
(assuming no corruption of frames)\n\n");
for(i=0; i<n; i+=w) {
for(j=i; j<i+w && j<n; j++) printf("%d ", f[j]);
printf("\nAcknowledgement of above frames sent is received by sender\n\n");}
return 0;
}
```

Output:

```
Enter window size: 4
Enter number of frames to transmit: 6
Enter 6 frames: 12 13 124 35 46 57
With sliding window protocol the frames will be sent in the following manner
(assuming no corruption of frames)

12 13 124 35
Acknowledgement of above frames sent is received by sender

46 57
Acknowledgement of above frames sent is received by sender
```

EXPERIMENT-8

AIM:Develop a Program to implement Stop and Wait Protocol.

Program:

```
#include <stdio.h>
int main() {
int i, n;
printf("Enter number of packets: ");
scanf("%d", &n);
for(i=1; i<=n; i++) {
printf("SENDER : Info = %d  Seq No = %d\n", i, (i-1)%2);
printf("RECEIVER : Packet %d received, Ack Sent\n", i);
printf("SENDER : Ack received\n");}
printf("DISCONNECTED\n");
return 0;
}
```

Output:

```
Enter number of packets: 6
SENDER : Info = 1  Seq No = 0
RECEIVER : Packet 1 received, Ack Sent
SENDER : Ack received
SENDER : Info = 2  Seq No = 1
RECEIVER : Packet 2 received, Ack Sent
SENDER : Ack received
SENDER : Info = 3  Seq No = 0
RECEIVER : Packet 3 received, Ack Sent
SENDER : Ack received
SENDER : Info = 4  Seq No = 1
RECEIVER : Packet 4 received, Ack Sent
SENDER : Ack received
SENDER : Info = 5  Seq No = 0
RECEIVER : Packet 5 received, Ack Sent
SENDER : Ack received
SENDER : Info = 6  Seq No = 1
RECEIVER : Packet 6 received, Ack Sent
SENDER : Ack received
DISCONNECTED
```

EXPERIMENT-9

AIM: Develop a program for congestion control using leaky bucket algorithm.

Program:

```
#include <stdio.h>
int main()
{
int b, r, p, n=0, i=1, ch, sent, drop;
printf("enter bucket size : ");
scanf("%d",&b);
printf("\nenter output rate :");
scanf("%d",&r);
printf("\nsecond\treceived\tsent\tdropped\tremained\n");
do
{
printf("\nenter number of packets coming at second %d :",i);
scanf("%d",&p);
n += p;
drop = n>b ? n-b : 0;
if(n>b) n=b;
sent = n<r ? n:r;
n -= sent;
printf("\nenter 1 to continue or 0 to quit ..... ");
scanf("%d",&ch);
printf("\n%d\t%d\t%d\t%d\t%d\n",i,p,sent,drop,n);
i++;}
while(ch==1);
return 0
;}
```

Output:

```
enter bucket size : 10

enter output rate :5

second  received    sent    dropped remained

enter number of packets coming at second 1 :2

enter 1 to continue or 0 to quit ........ 1

1   2   2   0   0

enter number of packets coming at second 2 :2

enter 1 to continue or 0 to quit ........ 1

2   2   2   0   0

enter number of packets coming at second 3 :1

enter 1 to continue or 0 to quit ........ 0

3   1   1   0   0
```

EXPERIMENT-10

AIM: Develop a Program to implement Dijkstra's algorithm to compute the Shortest path through a graph.

Program:

```
#include <stdio.h>
#define N 20
#define INF 999
int main()
{
int g[N][N],d[N],v[N]={0},n,s,i,j,u,min;
printf("Enter the number of vertices: ");
scanf("%d",&n);
printf("Enter the adjacency matrix (0 for no edge):\n");
for(i=0;i<n;i++)
for(j=0;j<n;j++) scanf("%d",&g[i][j]);
printf("Enter the source vertex: ");
scanf("%d",&s);
for(i=0;i<n;i++) d[i]=INF;
d[s]=0;
for(i=0;i<n;i++)
{
min=INF; u=-1;
for(j=0;j<n;j++)
if(!v[j] && d[j]<min) min=d[j],u=j;
if(u==-1) break;
v[u]=1;
for(j=0;j<n;j++)
if(g[u][j]>0 && !v[j] && d[u]+g[u][j]<d[j])
```

```
d[j]=d[u]+g[u][j];
}
printf("Vertex Distance from Source\n");
for(i=0;i<n;i++)
if(d[i]==INF) printf("%d INF\n",i);
else printf("%d %d\n",i,d[i]);
return 0;
}
```

Output:

```
Enter the number of vertices: 3
Enter the adjacency matrix (0 for no edge):
1 2 3
4 5 6
7 8 9
Enter the source vertex: 1
Vertex Distance from Source
0 4
1 0
2 6
```

EXPERIMENT-11

AIM: Develop a program to implement Distance Vector Routing Algorithm by obtaining routing table at each node(Take an Example subnet graph with weights indicating delay between).

Program:

```
#include <stdio.h>
#define N 20
#define INF 999
int main() {
int n,c[N][N],d[N][N],i,j,k;
printf("Enter the number of nodes :\n");
scanf("%d",&n);
printf("Enter the cost matrix :\n");
for(i=0;i<n;i++)
for(j=0;j<n;j++) {
scanf("%d",&c[i][j]);
d[i][j]=c[i][j]; }
for(k=0;k<n;k++)
for(i=0;i<n;i++)
for(j=0;j<n;j++)
if(d[i][j]>c[i][k]+d[k][j])
d[i][j]=c[i][k]+d[k][j];
for(i=0;i<n;i++) {
printf("For router %d\n",i+1);
for(j=0;j<n;j++)
printf("node %d via %d Distance %d\n",j+1,
i==j?i+1:j+1,d[i][j]);
}
```

```
return 0;
}
```

Output:

```
Enter the number of nodes :
3
Enter the cost matrix :
3 1 2
6 5 4
8 7 9
For router 1
node 1 via 1 Distance 3
node 2 via 2 Distance 1
node 3 via 3 Distance 2
For router 2
node 1 via 1 Distance 6
node 2 via 2 Distance 5
node 3 via 3 Distance 4
For router 3
node 1 via 1 Distance 8
node 2 via 2 Distance 7
node 3 via 3 Distance 9
```

EXPERIMENT-12

AIM:Develop a Program to Implement a Broadcast Tree by Taking a Subnet of Hosts.

Program:

```
#include <stdio.h>
#define N 10
#define INF 999
int main()
{
int n,a[N][N],v[N]={0},i,j,k,x,y,min,cost=0;
printf("Enter the number of nodes");
scanf("%d",&n);
printf("Enter the cost matrix\n");
for(i=0;i<n;i++)
for(j=0;j<n;j++) scanf("%d",&a[i][j]);
v[0]=1;
printf("Minimum spanning tree is\n");
for(i=1;i<n;i++)
{
min=INF;
for(j=0;j<n;j++) if(v[j])
for(k=0;k<n;k++)
if(!v[k] && a[j][k]>0 && a[j][k]<min)
min=a[j][k],x=j,y=k;
if(min==INF) break;
```

```
v[y]=1; cost+=min;
printf("%c %c %d\n",'A'+x,'A'+y,min);
}
printf("Minimum cost is %d\n",cost);
return 0;
}
```

Output:

```
Enter the number of nodes4
Enter the cost matrix
1 3 5 6
2 5 8 9
3 6 7 4
1 2 3 7
Minimum spanning tree is
A B 3
A C 5
C D 4
Minimum cost is 12
```

Experiment-13

Wireshark

i. **Packet Capture Using Wire shark**
ii. **Starting Wire shark**
iii. **Viewing Captured Traffic**
iv. **Analysis and Statistics & Filters.**

Title of the Experiment:

Network Traffic analysis using Wireshark tool.

Objectives:

To observe the Wireshark tool.

Theory:

Wireshark is a network protocol analyzer used to capture and inspect data packets in real-time. It helps in monitoring, troubleshooting, and analyzing network traffic by providing detailed information about packet headers, protocols, and data flow.

Procedure:

Installation of Wireshark:

- ✓ Open a browser, search "Download Wireshark", and choose the installer (32-bit or 64-bit).
- ✓ Run the installer and follow instructions to accept the license.
- ✓ Wireshark is ready to use.
- ✓ Linux users: Install via package repository.
- ✓ Check your network interface under Network & Internet settings.

i. Packet Capture Using Wireshark:

- ✓ Open Capture | Interfaces and select the active interface.
- ✓ Click Start to begin capturing packets.
    - ✓ Reproduce the network activity/problem. Avoid running other applications.
- ✓ Click Stop when done.
- ✓ Save the capture file in .pcap format via File →Save As.

ii. Starting Wireshark from Command Line:

- ✓ Run wireshark -h to see options.
- ✓ Example: wireshark -i <interface> to capture packets on a specific interface.
- ✓ Supports capture filters via -f option.

iii. Viewing Captured Traffic:

- ✓ Open a capture file or live capture.
- ✓ Click a packet in the packet list pane to view details in tree view and byte view.

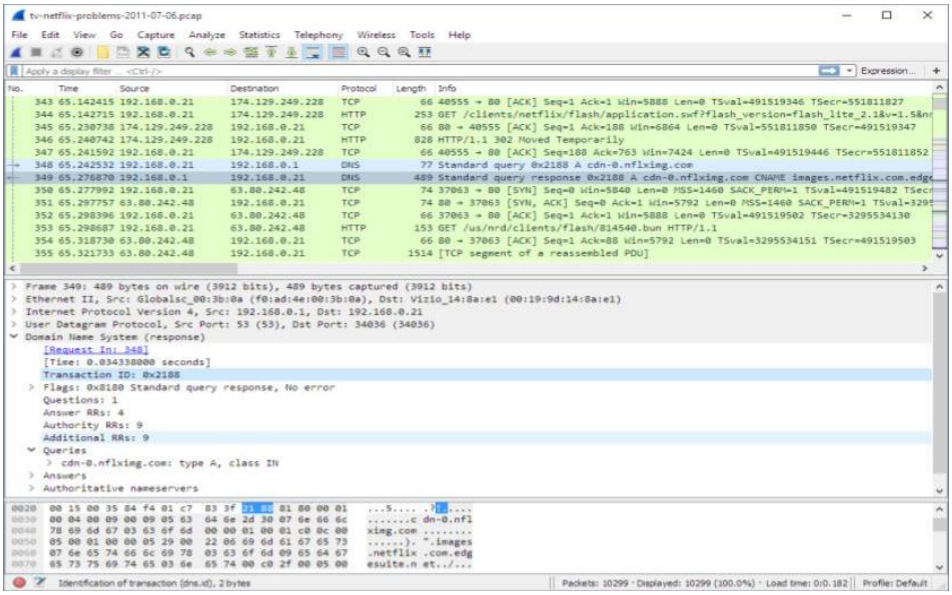
iv. Analysis, Statistics & Filters:

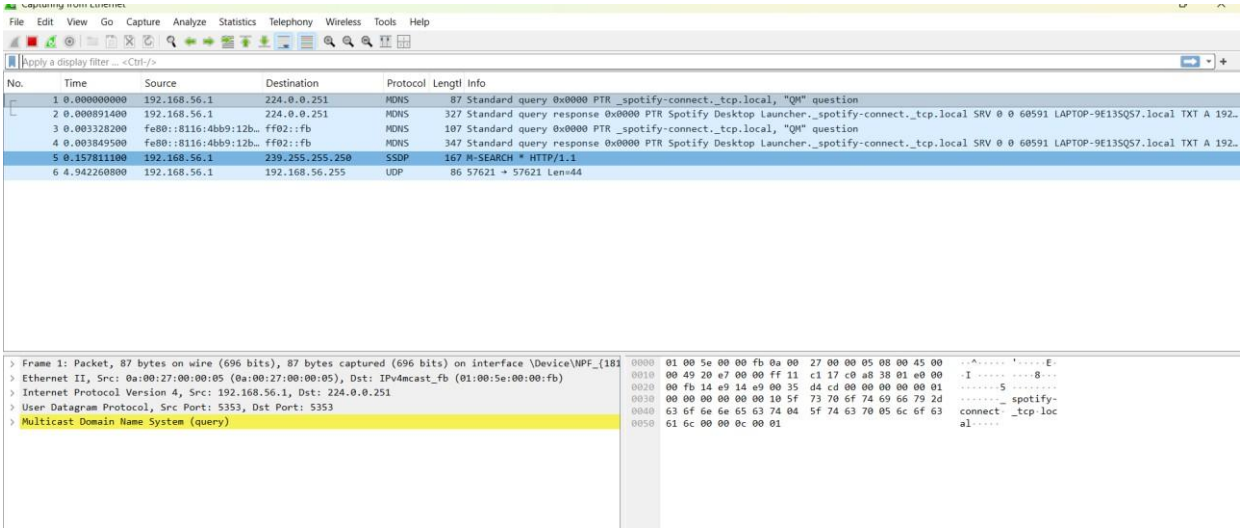
- ✓ Analyze Menu: Tools to inspect packet details.
- ✓ Statistics Menu: Shows protocol hierarchy, endpoints, conversations, I/O graphs, etc.

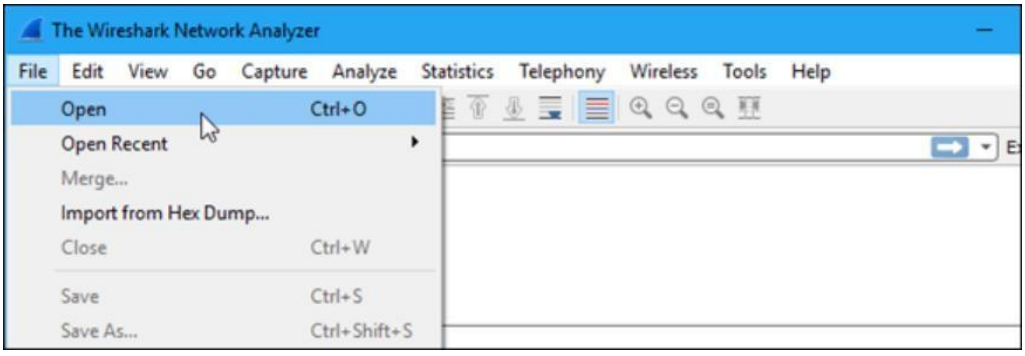
Filters:

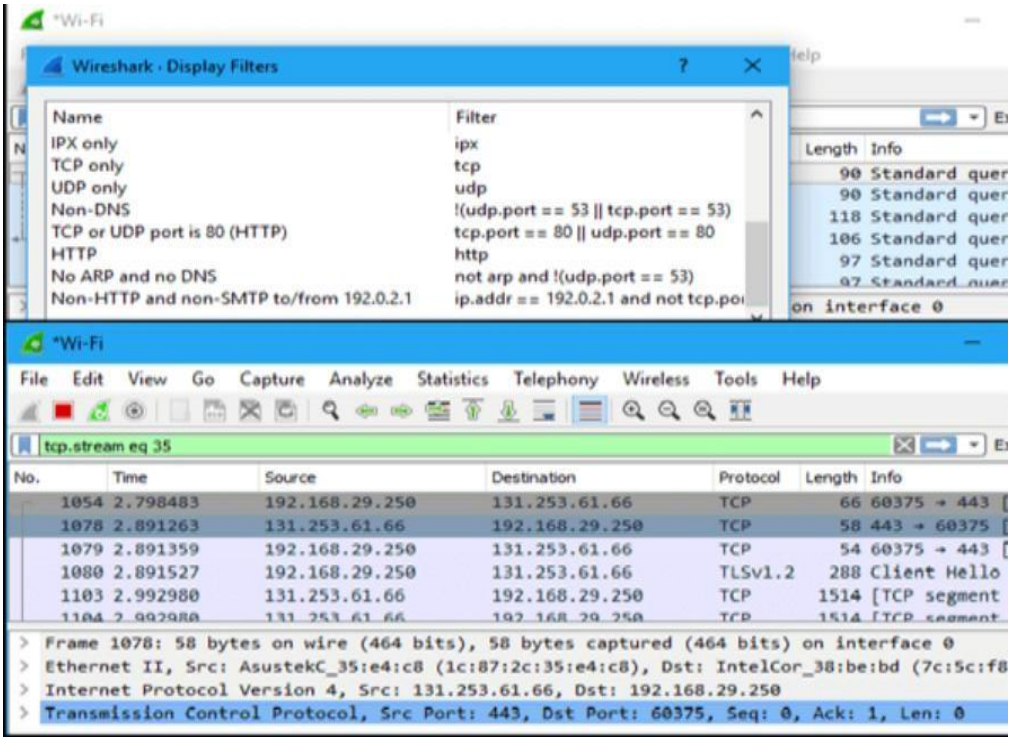
- Capture filters: Apply during packet capture.
- Display filters: Apply to packets shown after capture.

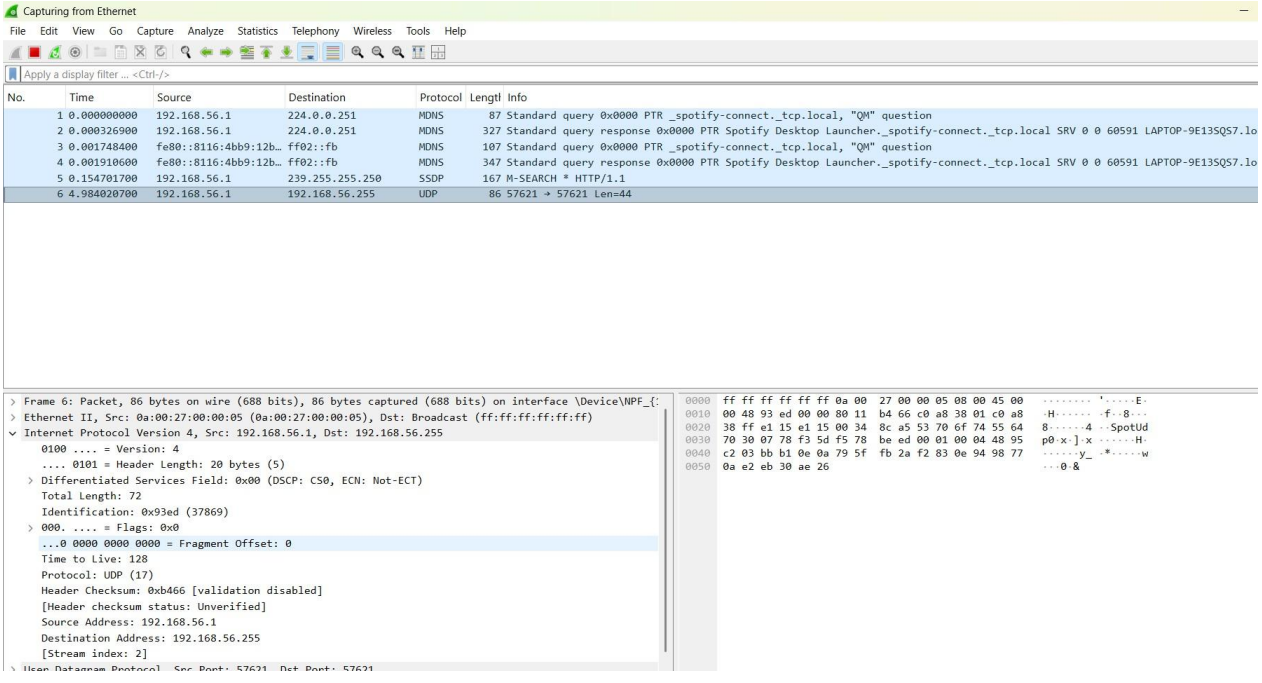
Screenshots:








Wireshark · Display Filters
Name
Filter
IPX only
ipx
TCP only
tcp
UDP only
udp
Non-DNS
!(udp.port == 53 || tcp.port == 53)
TCP or UDP port is 80 (HTTP)
tcp.port == 80 || udp.port == 80
HTTP
http
No ARP and no DNS
not arp and !(udp.port == 53)
Non-HTTP and non-SMTP to/from 192.0.2.1
ip.addr == 192.0.2.1 and not tcp.po
*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.stream eq 35
No. Time Source Destination Protocol Length Info
1054 2.798483 192.168.29.250 131.253.61.66 TCP 66 60375 → 443
1078 2.891263 131.253.61.66 192.168.29.250 TCP 58 443 → 60375
1079 2.891359 192.168.29.250 131.253.61.66 TCP 54 60375 → 443
1080 2.891527 192.168.29.250 131.253.61.66 TLSv1.2 288 Client Hello
1103 2.992980 131.253.61.66 192.168.29.250 TCP 1514 [TCP segment
Frame 1078: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface 0
Ethernet II, Src: AsustekC_35:e4:c8 (1c:87:2c:35:e4:c8), Dst: IntelCor_38:be:bd (7c:5c:f8
Internet Protocol Version 4, Src: 131.253.61.66, Dst: 192.168.29.250
Transmission Control Protocol, Src Port: 443, Dst Port: 60375, Seq: 0, Ack: 1, Len: 0


Capturing from Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 192.168.56.1 224.0.0.251 MDNS 87 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
2 0.000326900 192.168.56.1 224.0.0.251 MDNS 327 Standard query response 0x0000 PTR Spotify Desktop Launcher._spotify-connect._tcp.local SRV 0 0 60591 LAPTOP-9E13SQS7.lo
3 0.001748400 fe80::8116:4bb9:12b… ff02::fb MDNS 107 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
4 0.001910600 fe80::8116:4bb9:12b… ff02::fb MDNS 347 Standard query response 0x0000 PTR Spotify Desktop Launcher._spotify-connect._tcp.local SRV 0 0 60591 LAPTOP-9E13SQS7.lo
5 0.154701700 192.168.56.1 239.255.255.250 SSDP 167 M-SEARCH * HTTP/1.1
6 4.984020700 192.168.56.1 192.168.56.255 UDP 86 57621 → 57621 Len=44
Frame 6: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF_{
Ethernet II, Src: 0a:00:27:00:00:05 (0a:00:27:00:00:05), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Internet Protocol Version 4, Src: 192.168.56.1, Dst: 192.168.56.255
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 72
Identification: 0x93ed (37869)
000. .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: UDP (17)
Header Checksum: 0xb466 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.56.1
Destination Address: 192.168.56.255
[Stream index: 2]
0000 ff ff ff ff ff ff 0a 00 27 00 00 05 08 00 45 00
0010 00 48 93 ed 00 00 80 11 b4 66 c0 a8 38 01 c0 a8
0020 38 ff e1 15 e1 15 00 34 8c a5 53 70 6f 74 55 64
0030 70 30 07 78 f3 5d f5 78 be ed 00 01 00 04 48 95
0040 c2 03 bb b1 0e 0a 79 5f fb 2a f2 83 0e 94 98 77
0050 0a e2 eb 30 ae 26

Result:

Successfully installed Wireshark on the computer. Captured live network traffic on the selected interface.

Viewed packet details including source, destination, protocol, and payload. Applied filters to focus on specific traffic and used statistics to analyze network behavior.

Conclusion:

Wireshark is a powerful tool for monitoring and analyzing network traffic. It helps in identifying network issues, understanding protocol behavior, and inspecting data flow. Using packet capture, filters, and statistics, we can efficiently study and troubleshoot network performance.

Experiment-14

Discover active hosts, open ports and running services in a network using Nmap.

Title of the Experiment:

Discover active hosts, open ports and running services in a network using Nmap.

Objectives:

To discover active hosts, open ports and running services in a network using Nmap.

Theory:

Nmap is a network scanning tool used to discover active hosts, open ports, and running services in a network. It helps in network monitoring, security auditing, and troubleshooting by mapping devices and detecting services efficiently.

Procedure:

Install Nmap:

- **Windows:** Download from nmap.org and install.
- **Linux:** Use `sudo apt install nmap` (Ubuntu/Debian) or `sudo yum install nmap` (CentOS).

Discover Active Hosts:

- **Command:**

```
nmap -sn 192.168.1.0/24
```

- **Explanation:** `-sn` (ping scan) lists active hosts in the subnet.

Scan for Open Ports:

Command:

```
nmap 192.168.1.5
```

Explanation: Scans the specified host for open TCP ports.

Detect Running Services:

Command:

```
nmap -sV 192.168.1.5
```

Explanation: `-sV` detects service versions running on open ports.

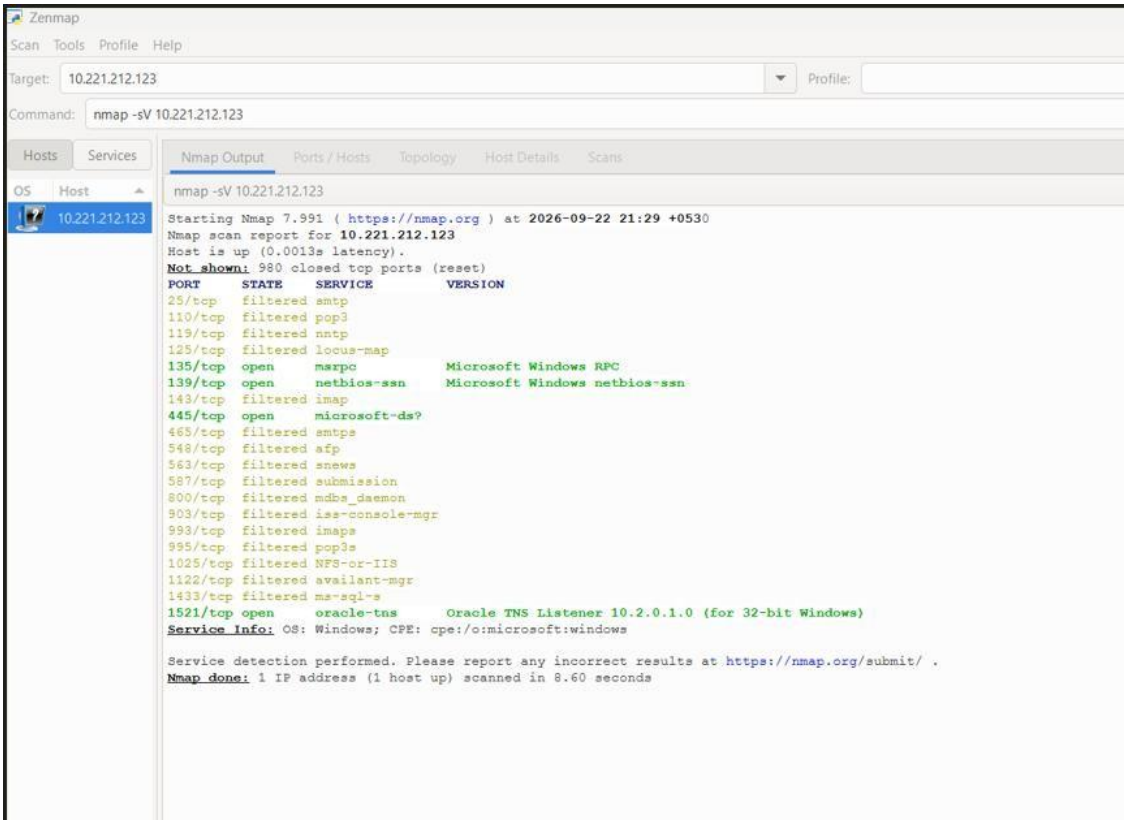
Scan Multiple Hosts or Subnets:

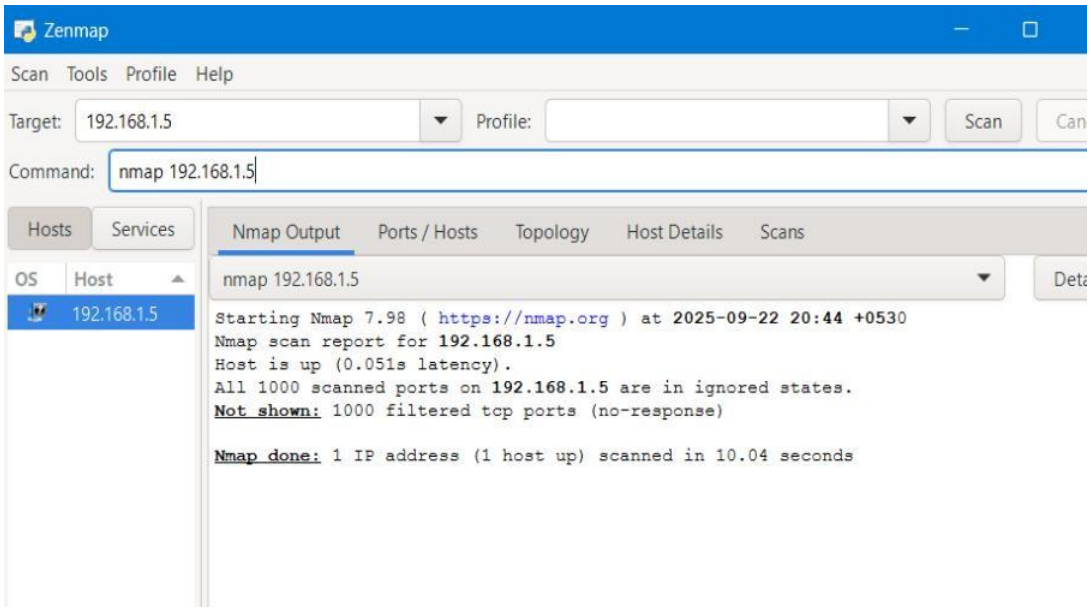
Command:

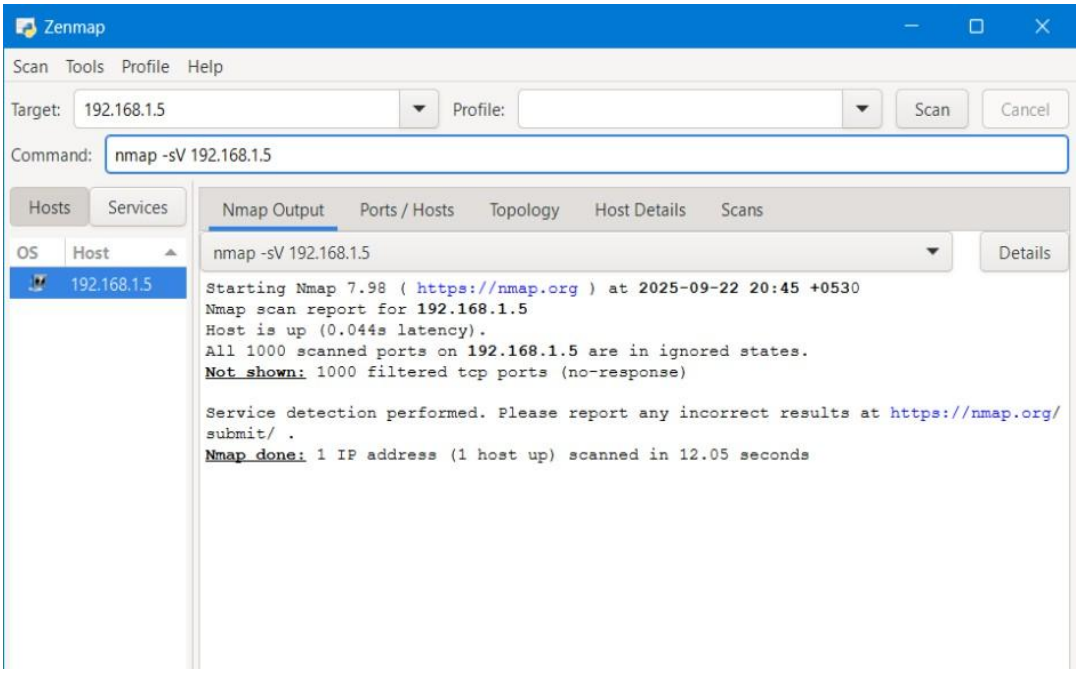
```
nmap -p 1-1000 192.168.1.0/24
```

Explanation: Scans ports 1 to 1000 for all hosts in the subnet.

Screenshots:







Result:

Active hosts in the network were discovered.

Open ports and running services on hosts were identified.

Network information was successfully captured and displayed.

Conclusion:

Nmap is an effective tool for network scanning and service discovery, helping in network monitoring, troubleshooting, and security analysis.

Experiment-15

Find Operating System in a host using Nmap.

Title of the Experiment:

Find Operating System in a host using Nmap.

Objectives:

To find Operating System in host using Nmap.

Theory:

Nmap can identify the operating system of a host by analyzing network responses and TCP/IP stack characteristics. This helps in network auditing, security assessment, and troubleshooting by knowing what OS each host is running.

Procedure:

Open Command Prompt (`Win + R →cmd →Enter`).

Run Nmap OS detection command.

```
nmap -O <target_IP>
```

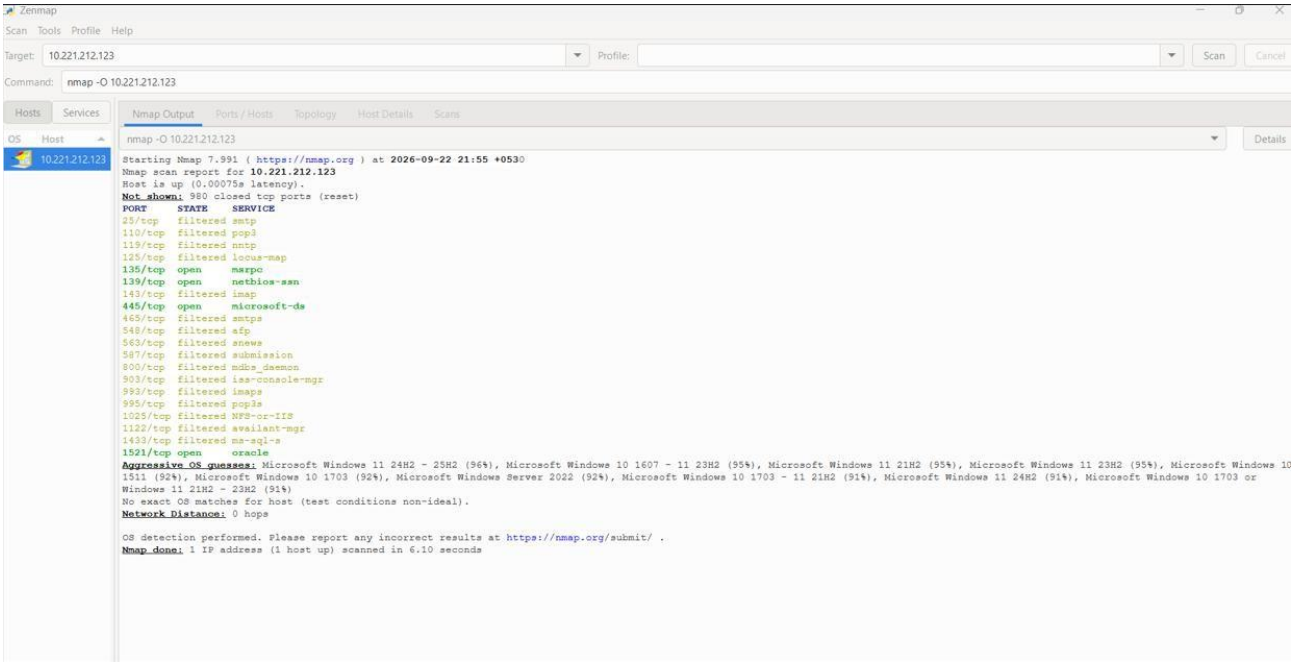
Replace `<target_IP>` with the host's IP address.

View the output: Nmap will display the detected operating system and version if possible.

Optional: Save the results:

```
nmap -O <target_IP> -oN os_result.txt
```

Screenshots:



Results:

Host `maa05s06-in-x0e.1e100.net` is active.

Open ports detected: 80 (HTTP) and 443 (HTTPS).

998 ports were filtered, likely by a firewall.

Conclusion:

Nmap successfully identified the active host, open ports, and services, demonstrating its usefulness in network scanning, monitoring, and security analysis.

Experiment-16

16. Do the Following Using NS2 Simulator:

i. NS2 Simulator – Introduction

ii. Simulate to Find the Number of Packets Dropped.

iii. Simulate to Find the Number of Packets Dropped by TCP/UDP.

iv. Simulate to Find the Number of Packets Dropped Due to Congestion.

v. Simulate to Compare Data Rate and Throughput.

Title of the Experiment:

Simulation of Network Performance Parameters using NS2 Simulator

Objectives:

1. To understand the basic working of the **NS2 (Network Simulator 2)**.
2. To create and simulate a simple network using NS2.
3. To find the **number of packets dropped** during transmission.
4. To analyze packets dropped by **TCP and UDP**.
5. To study packet drops caused by **network congestion**.
6. To compare **data rate and throughput** of a network.

Theory:

NS2 (Network Simulator 2) is an open-source, discrete-event network simulator used to simulate and analyze computer networks.

- NS2 supports protocols such as **TCP, UDP, IP, FTP, and CBR**.
- A network is created using **nodes, links, agents, and applications**.
- **TCP** provides reliable data transmission and congestion control.
- **UDP** provides faster transmission but does not guarantee delivery.
- **Packet drop** occurs when packets cannot be delivered, commonly because of congestion or limited buffer space.
- **Throughput** is the amount of successfully delivered data per unit time.

The basic flow is:

Create Nodes → Create Links → Attach TCP/UDP → Generate Traffic → Run Simulation → Analyze Results

```
% puts "NS2 is working"
NS2 is working
% cd ~/ns2
% ls
exp2.nam  exp2.tcl  exp2.tr
% ns exp2.tcl
warning: using backward compatibility mode
error when calling class OldSim: exp2.tcl
% exit
purnima@LAPTOP-9E13SQS7:~/ns2$ ns exp2.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ nam:
^C
purnima@LAPTOP-9E13SQS7:~/ns2$ ls
exp2.nam  exp2.tcl  exp2.tr
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="d"{drop++} END{print "Packets Dropped:",drop}' exp2.tr
Packets Dropped: 1228
purnima@LAPTOP-9E13SQS7:~/ns2$ nam exp2.nam
nam:
purnima@LAPTOP-9E13SQS7:~/ns2$
```

purnima@LAPTOP-9E13SQS7:

```
  GNU nano 6.2
set ftp [new Application/FTP]
$ftp attach-agent $tcp

proc finish {} {
    global ns tf
    $ns flush-trace
    close $tf
    exit 0
}

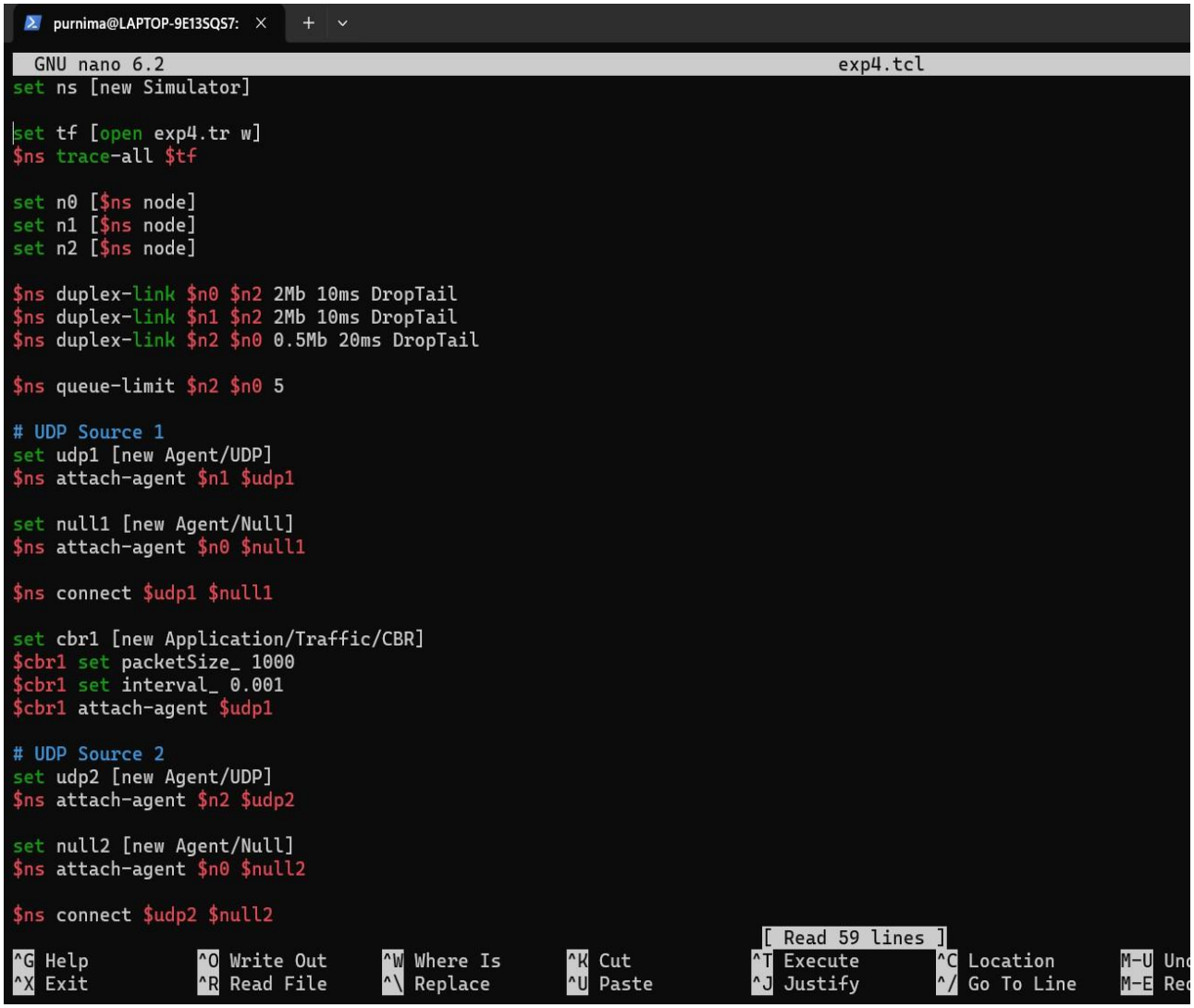
$ns at 0.5 "$cbr start"
$ns at 0.5 "$ftp start"

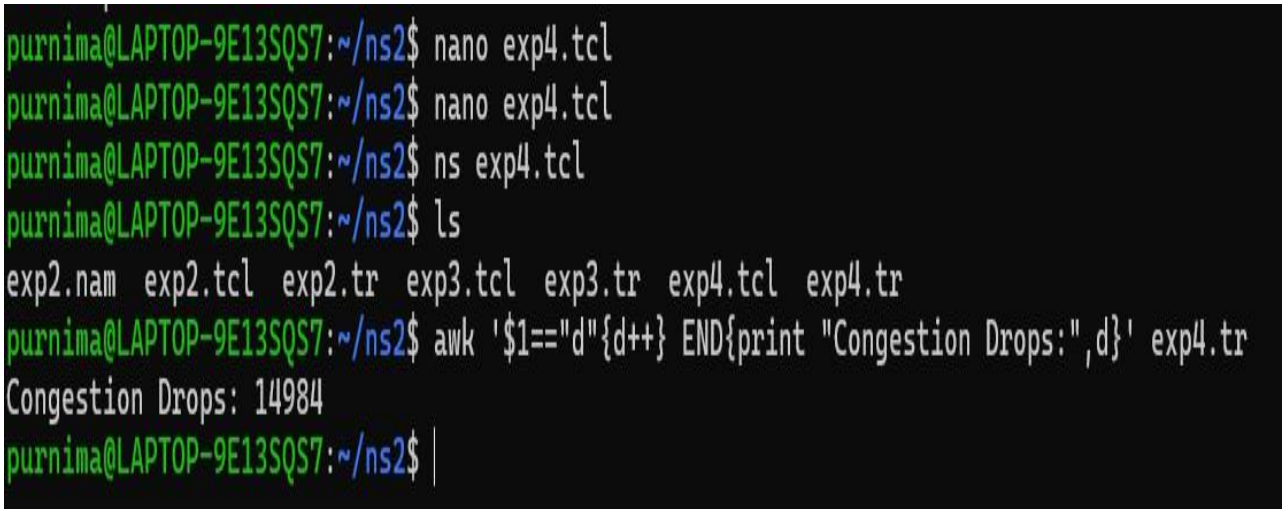
$ns at 9.5 "$cbr stop"
$ns at 9.5 "$ftp stop"

$ns at 10.0 "finish"

$ns run
```

```
purnima@LAPTOP-9E13SQS7:~/ns2$ ns exp3.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ ls
exp2.nam  exp2.tcl  exp2.tr  exp3.tcl  exp3.tr
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="d" && $5=="cbr"{n++} END{print "UDP Drops:",n}' exp3.tr
UDP Drops: 1229
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="d" && $5=="tcp"{n++} END{print "TCP Drops:",n}' exp3.tr
TCP Drops: 8
purnima@LAPTOP-9E13SQS7:~/ns2$
```


purnima@LAPTOP-9E13SQS7:
GNU nano 6.2 exp4.tcl
set ns [new Simulator]
set tf [open exp4.tr w]
$ns trace-all $tf
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns duplex-link $n2 $n0 0.5Mb 20ms DropTail
$ns queue-limit $n2 $n0 5
UDP Source 1
set udp1 [new Agent/UDP]
$ns attach-agent $n1 $udp1
set null1 [new Agent/Null]
$ns attach-agent $n0 $null1
$ns connect $udp1 $null1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 1000
$cbr1 set interval_ 0.001
$cbr1 attach-agent $udp1
UDP Source 2
set udp2 [new Agent/UDP]
$ns attach-agent $n2 $udp2
set null2 [new Agent/Null]
$ns attach-agent $n0 $null2
$ns connect $udp2 $null2
[Read 59 lines]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E


purnima@LAPTOP-9E13SQS7:~/ns2$ nano exp4.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ nano exp4.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ ns exp4.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ ls
exp2.nam exp2.tcl exp2.tr exp3.tcl exp3.tr exp4.tcl exp4.tr
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="d"{d++} END{print "Congestion Drops:",d}' exp4.tr
Congestion Drops: 14984
purnima@LAPTOP-9E13SQS7:~/ns2$




```
purnima@LAPTOP-9E13SQS7:~/ns2$ ns exp5.tcl
purnima@LAPTOP-9E13SQS7:~/ns2$ ls
exp2.nam  exp2.tcl  exp2.tr  exp3.tcl  exp3.tr  exp4.tcl  exp4.tr  exp5.tcl  exp5.tr
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="r" && $4=="cbr" && $3==1 {bytes += $6} END {print "Throughput:", bytes*8/9/1000, "Kbps"}' exp5.tr
Throughput: 0 Kbps
purnima@LAPTOP-9E13SQS7:~/ns2$
```

```
purnima@LAPTOP-9E13SQS7:~/ns2$ ls
exp2.nam  exp2.tcl  exp2.tr  exp3.tcl  exp3.tr  exp4.tcl  exp4.tr  exp5.tcl  exp5.tr
purnima@LAPTOP-9E13SQS7:~/ns2$ awk '$1=="r" && $4=="cbr" && $3==1 {bytes += $6} END {print "Throughput:", bytes*8/9/1000, "Kbps"}' exp5.tr
Throughput: 0 Kbps
purnima@LAPTOP-9E13SQS7:~/ns2$
```